
MILESTONE 1 — EXECUTION DOCUMENT

FreightQuote AI

Route Intelligence & Quotation Foundation — Weeks 1-2 Build Specification

DOCUMENT TYPE	Milestone execution spec — form fields, calculation methods, UI pages, backend modules, day plan
PROJECT CODE	FQ-AMB-001 / M1
DURATION	10 working days (Weeks 1-2)
EXIT CRITERIA	Route recommendations for $\geq 98\%$ of test lanes • Transit $MAE \leq 2.0$ days • Quotation dashboard functional
SCOPE NOTE	Full pricing engine is Milestone 2. M1 delivers weights, transit and an <i>indicative</i> total from a simple rate table.
AUDIENCE	Frontend devs, Python devs, ML dev, QA, Tech Lead

Table of Contents

1. What Milestone 1 Must Deliver

Scope boundary and exit criteria

2. Complete Form Field Specification

Every field, type, validation, consumer

3. Calculation Method Specification

Every method: input, output, formula, location

4. Reference Data Required

What must be loaded before code works

5. Frontend Build List

Pages, components, folder placement

6. Python Backend Build List

Modules, responsibilities, endpoints

7. Transit Time Model

Baseline vs ML, features, evaluation

8. Day-by-Day Plan

10 working days, task allocation

9. Definition of Done Checklist

Signed off before Milestone 2 starts

1. What Milestone 1 Must Deliver

1.1 The Scope Boundary

The single most common way a milestone like this fails is scope bleed — the team starts building the pricing engine because it feels connected. It is not in Milestone 1. Keep the line clear:

IN Milestone 1	NOT in Milestone 1 (Milestone 2 or later)
Port/airport master data loaded and searchable	Rate card management UI, rate card import
Shipment enquiry form, fully validated	Surcharge engine (BAF, CAF, THC, PSS)
Gateway resolution (city → port/airport)	Margin policy, margin floor, approval workflow
Distance calculation for all three legs	Quote versioning and immutability
Weight calculations (actual / volumetric / chargeable)	PDF quotation generation
Transit time estimation with a range	Weather Agent, Customs Agent, Margin Agent
Route Agent producing ranked route options	Composite risk scoring
Route map rendering, carrier comparison table	Customer portal, approval queue
Quotation dashboard (list, filter, detail)	Win probability model
Indicative total from a flat lane rate table	Accurate cost build-up honouring incoterm

Say this out loud to the team on day one

The Live Estimate panel will show a total in Milestone 1, and it will be roughly right — but it comes from a flat lane rate table, not a real pricing engine. Label it "Indicative" in the UI. If you let the team build real pricing now, the Route Agent and the transit model will not be finished by day 10, and the milestone fails on its actual exit criteria.

1.2 Exit Criteria and How Each Is Measured

Criterion	Measurement	Evidence required at review
Route recommendations generated	Run 200 test lanes across Asia-Europe, Trans-Pacific, Intra-Asia, Middle East	Script output: ≥ 98% of lanes return ≥ 2 viable routings
Transit time estimation operational	MAE on a hold-out set of historical shipments	Evaluation notebook + printed MAE, must be ≤ 2.0 days
Quotation dashboard functional	UAT walkthrough with the tech lead	Live demo: create enquiry → see routes on map → open detail → filter list

2. Complete Form Field Specification

This is the contract between the frontend and the backend. Every field below has a Zod schema entry, a serializer field, and a database column. Fields marked shaded do not exist in the current deployed form and must be added — without them the Customs and Margin agents cannot function in later milestones, and the pricing will be structurally wrong.

2.1 Section 1 — Route

Field	Control	Required	Validation	Consumed by
origin_gateway	Searchable dropdown (async)	Yes	Must resolve to a port/airport ID in master data. Free text rejected.	Route, Pricing, Customs
destination_gateway	Searchable dropdown (async)	Yes	Must differ from origin. Must be serviced for the chosen mode.	Route, Pricing, Customs
pickup_address	Textarea + PIN field	Conditional	Required when door pickup applies (incoterm EXW/FCA/DAP/DDP). PIN 6 digits for India.	Route (leg 1 distance)
delivery_address	Textarea + PIN field	Conditional	Required when door delivery applies (DAP/DDP).	Route (leg 3 distance)
ready_date	Date picker	Yes	≥ tomorrow, ≤ today + 180 days. Business day preferred.	Route, Weather (M3)
required_delivery_date	Date picker	No	> ready_date. Warn if unachievable by any routing.	Route (urgency weighting)

The single highest-value change in this section

The current form uses free-text inputs with placeholders like "e.g. Mumbai, Maharashtra". This must become an async searchable dropdown backed by the ports table. A user typing "bombay", "Mumbai Port", or "NSA" produces three different unresolvable strings. Once the user selects a structured gateway record, every downstream calculation becomes deterministic. Build this on day 2 — everything else depends on it.

2.2 Section 2 — Service Type

Field	Control	Required	Validation	Consumed by
mode	Segmented buttons	Yes	OCEAN / AIR / GROUND_RAIL / EXPRESS_AIR. Default OCEAN.	All agents
load_type	Radio, shown only when mode = OCEAN	Conditional	FCL / LCL. Required for ocean. Drives the entire weight model.	Pricing, Route
incoterm	Dropdown	Yes	EXW / FCA / FOB / CIF / CFR / DAP / DDP. Default FOB.	Pricing, Customs

Why incoterm is not optional: it determines which cost legs belong in the quote at all. Under EXW the seller does nothing and every leg is chargeable; under DDP the quote must include destination duties and delivery. A quote produced without knowing the incoterm is not a quote — it is a number with no defined scope. It also drives whether `pickup_address` and `delivery_address` are required.

2.3 Section 3 — Shipment Details (repeatable line items)

The fields shown depend on `package_type`. This conditional branch is the difference between a form that works and one that produces nonsense.

Common to every line item

Field	Control	Required	Validation	Consumed by
package_type	Dropdown	Yes	CONTAINER / PALLET / CARTON / CRATE / DRUM / BAG / LOOSE	Pricing (branch selector)
commodity_description	Text	Yes	Min 3 characters. "General cargo" rejected — too vague for customs.	Customs, Pricing
hs_code	Text with autocomplete	No	6-8 digits. Suggested from commodity_description.	Customs (duty, restrictions)

When package_type = CONTAINER (FCL path)

Field	Control	Required	Validation	Consumed by
container_type	Dropdown	Yes	20GP / 40GP / 40HC / 20RF / 40RF / 20OT / 40FR	Pricing, Route (carrier capability)

container_count	Number stepper	Yes	1-50, integer	Pricing
gross_weight_kg	Number	Yes	> 0. Warn above payload limit for the type (20GP ≈ 28,000 kg).	Pricing (limit check only)

Note the absence of dimensions. FCL is priced per container as a flat rate. Volumetric weight is meaningless. The current form shows Length/Width/Height under a "Container" package type — that combination should never appear.

When package_type is anything else (LCL / Air path)

Field	Control	Required	Validation	Consumed by
quantity	Number stepper	Yes	1-9999, integer	Weight calculation
weight_per_unit_kg	Number	Yes	> 0, max 5000 per piece	Weight calculation
length_cm / width_cm / height_cm	Three number inputs	Yes	Each > 0. Warn if any dimension exceeds air ULD limits (317 cm).	Volumetric weight
is_stackable	Checkbox	No	Default true. Affects LCL consolidation.	Pricing (M2)

2.4 Section 4 — Additional Details

Field	Control	Required	Validation	Consumed by
declared_value	Number + currency select	Conditional	Required when insurance is selected. > 0.	Margin, Insurance calc
currency	Dropdown	Yes	INR / USD / EUR / AED. Default INR.	Pricing, FX freeze
is_fragile	Checkbox	No	—	Pricing (surcharge)
is_hazardous	Checkbox	No	Reveals three more fields when ticked.	Customs, Route (carrier filter)
un_number	Text, shown when hazardous	Conditional	Format UN0000-UN3999. Required when hazardous.	Customs, Route
imo_class	Dropdown, shown when hazardous	Conditional	Classes 1-9 with sub-divisions.	Customs, Route
msds_file	File upload, shown when hazardous	Conditional	PDF only, max 10 MB, virus scanned.	Customs
is_temperature_controlled	Checkbox	No	Reveals temperature range. Forces reefer container types.	Route, Pricing
temp_min_c / temp_max_c	Two numbers	Conditional	−30 to +30. min < max.	Route (equipment)
needs_insurance	Checkbox	No	Makes declared_value required.	Pricing
special_instructions	Textarea	No	Max 500 characters. Treated as untrusted text by agents.	All agents (context only)

2.5 Section 5 — Contact Details

Field	Control	Required	Validation	Consumed by
full_name	Text	Yes	2-100 characters	CRM
company_name	Text	Yes	2-150 characters	Margin (tier lookup)
email	Email	Yes	RFC-valid, MX check optional	Notifications
phone	Country code + number	Yes	E.164 format	CRM
country	Dropdown	Yes	ISO 3166-1 alpha-2	Customs, Margin
existing_customer_code	Text, behind a toggle	No	If provided, must resolve to a customer record.	Margin (tier & contract rates)

2.6 Field Behaviour Rules

Rule	Implementation
Validate on blur, not on submit	Zod resolver with mode: 'onBlur'. Showing eight errors after submit is the worst pattern in form design.
Hide conditional fields entirely	Do not disable — unmount. A shorter form gets completed.
Never ask for derivable data	Origin country from gateway. Volumetric weight from dimensions. Chargeable weight computed. Region from country.

Smart defaults on load	quantity 1, currency INR, ready_date tomorrow, mode OCEAN, incoterm FOB, quantity stepper focused.
Draft persistence	Debounced write to localStorage every 2 s. On return show "Continue where you left off".
Progress indicator	Step badges 1-5 become clickable anchors; completed steps show a tick.
Mobile layout	Two-column pairs collapse to single column below 768 px. Live Estimate becomes a fixed bottom bar showing the total, expandable on tap.

3. Calculation Method Specification

Every method below is a **pure function**: no database access, no HTTP calls, no React. Given the same input it returns the same output. This makes them testable in isolation and lets the identical logic run in the browser for the Live Estimate and on the server for verification.

3.1 Method Index

Method	Module	Returns
resolve_gateway	gateway.py / gateway.ts	Gateway record for a city and mode
haversine_distance	distance.py	Great-circle km between two coordinates
road_distance	distance.py	Estimated road km
sea_distance	distance.py	Nautical miles from the port distance table
main_leg_distance	distance.py	Distance for the chosen mode
actual_weight	weight.py	Total physical weight in kg
volumetric_weight	weight.py	Dimensional weight in kg
chargeable_weight	weight.py	The greater of the two, or R/T for LCL
estimate_transit	transit.py	Transit day range with a component breakdown
add_business_days	transit.py	Calendar-aware arrival date
build_route_options	route_agent.py	Ranked list of viable routings
score_route	route_agent.py	Composite ranking score 0-1
indicative_total	pricing_stub.py	M1 placeholder total from a flat lane table
compute_live_estimate	index.py / index.ts	The whole panel payload

3.2 Gateway Resolution

Purpose: convert a user-selected city or a raw coordinate into a serviceable port, airport or rail hub for the chosen mode.

```
resolve_gateway(city_or_point, mode) → Gateway
target_type = PORT if mode == OCEAN
AIRPORT if mode in (AIR, EXPRESS_AIR)
HUB if mode == GROUND_RAIL
candidates = gateways where type == target_type and is_active and mode in supported_modes
return min(candidates, key = haversine_distance(point, candidate))
```

If the user selected a gateway directly from the dropdown, skip resolution entirely and use it. Resolution only runs when the input is a city (customer portal or EDI ingestion). Cache results by city — the same lookup will repeat thousands of times.

3.3 Distance Methods

haversine_distance(lat1, lon1, lat2, lon2) → km

```
φ1, φ2 = radians(lat1), radians(lat2)
Δφ = radians(lat2 - lat1)
Δλ = radians(lon2 - lon1)
a = sin²(Δφ/2) + cos(φ1) · cos(φ2) · sin²(Δλ/2)
c = 2 · atan2(√a, √(1-a))
distance = 6371 · c
```

road_distance(point_a, point_b, country) → km

```
road_km = haversine_distance(a, b) × DETOUR_FACTOR[country]
DETOUR_FACTOR = { IN: 1.30, AE: 1.20, EU: 1.25, US: 1.22, DEFAULT: 1.30 }
```

A road-distance API gives better numbers, but the detour factor is accurate to roughly 10% and costs nothing. Use the factor in M1; make the function swappable so an API can replace it later without touching callers.

sea_distance(origin_port, destination_port) → nautical miles

```
row = port_distances.lookup(origin_port.code, destination_port.code)
if row exists: return row.nautical_miles
else: return NULL → lane not serviced, route rejected
```

Never use haversine for the ocean leg
Great-circle distance cuts straight across land. Mumbai to Rotterdam is about 6,400 km in a straight line and about 11,300 km by sea via Suez — an error of 76%. Every transit estimate and every rate built on it would be wrong. Use a seeded port-to-port distance table. For Milestone 1, populate the top 50 lanes manually from published tables; that is a two-hour data task and it removes the largest single source of error in the system.

main_leg_distance(origin_gw, dest_gw, mode) → distance

Mode	Method
OCEAN	sea_distance() lookup. No fallback — a missing lane means no routing.
AIR / EXPRESS_AIR	haversine × 1.06 (airway routing overhead)
GROUND_RAIL	haversine × 1.30 road, or × 1.20 where a rail corridor exists

3.4 Weight Methods

actual_weight(items) → kg

```
sum over items of (weight_per_unit_kg × quantity)
```

volumetric_weight(items, mode) → kg

```
sum over items of (length_cm × width_cm × height_cm × quantity) / DIVISOR[mode]
DIVISOR = { AIR: 6000, EXPRESS_AIR: 5000, GROUND_RAIL: 4500, OCEAN_LCL: 1000 }
```

The ocean LCL divisor of 1000 converts cm³ to CBM rather than to kg — ocean LCL is charged on revenue tons, not dimensional weight. Handle it as a separate branch rather than pretending it is the same calculation.

chargeable_weight(items, mode, load_type) → ChargeableResult

```
if package_type == CONTAINER (FCL): return { basis: "PER_CONTAINER", units: container_count, chargeable: null } ← weight is a LIMIT, not a charge basis
elif mode == OCEAN and load_type == LCL: cbm = volume_cm3 / 1_000_000 tonnes = actual_weight / 1000
return { basis: "REVENUE_TON", units: max(cbm, tonnes) }
else: (air, express, ground) return { basis: "CHARGEABLE_KG", units: max(actual_weight, volumetric_weight) }
```

Three genuinely different charge bases. The current form's Live Estimate always shows "Chargeable weight", which is only meaningful for the third branch. For FCL it should read "Containers: 2 × 40HC"; for LCL, "Revenue tons: 4.2 R/T".

3.5 Transit Time Estimation

estimate_transit(route, mode, ready_date) → TransitResult

```
T_pickup = ceil(pickup_road_km / 450) min 1, 0 if not door pickup
T_origin = ORIGIN_DWELL[mode]
T_linehaul = main_leg_distance / SPEED[mode]
T_schedule = (7 / sailings_per_week) / 2 ocean only, average wait
T_dest = DEST_DWELL[mode]
T_delivery = ceil(delivery_road_km / 450) min 1, 0 if not door delivery
T_total = sum of the above range = [T_total, T_total + BUFFER[mode]]
arrival = add_business_days(ready_date, T_total, origin_cal, dest_cal)
```

Mode constants

Mode	Line-haul speed	Origin dwell	Dest dwell	Buffer
Ocean FCL	400 nm/day	3 d	3 d	4 d
Ocean LCL	400 nm/day	5 d	5 d	5 d
Air Freight	800 km/h + 1 d handling	2 d	1.5 d	2 d
Express Air	800 km/h, priority	0.5 d	0.5 d	1 d
Ground & Rail	450 km/day road, 600 rail	0.5 d	0.5 d	2 d

Dwell times cover cutoff, consolidation, terminal handling and customs clearance. LCL dwell is longer at both ends because cargo waits for consolidation and deconsolidation.

add_business_days(start, days, origin_calendar, dest_calendar) → date

Step forward day by day, skipping weekends and holidays for the relevant calendar. A Friday pickup into a Sunday-closed port is not a two-day transit. India, UAE and European calendars differ — UAE weekends are Friday-Saturday in some contexts. Store a per-country calendar table.

3.6 Route Agent

build_route_options(shipment) → List[RouteOption]

1. Resolve origin and destination gateways for the selected mode.
2. Query carrier services covering that gateway pair. Include direct services and one-transshipment paths.
3. Filter by capability: reefer if temperature controlled, hazmat acceptance if hazardous, container type availability, capacity for the volume.
4. For each surviving service, compute distance, transit and congestion at both ends.
5. Score and rank. Return the top 3, flagging the highest scorer as recommended.
6. If fewer than 2 options survive, widen to two-transshipment paths. If still none, return a structured "lane not serviced" result — never an empty list with no explanation.

score_route(route, shipment) → 0..1

```
score = 0.35 × transit_score (faster is better, normalised across options) + 0.30 × cost_score (cheaper is better) + 0.20 × reliability_score (carrier on-time percentage) + 0.15 × congestion_score (lower port congestion is better) If required_delivery_date is set and the route misses it: score = score × 0.4 heavy penalty, still shown with a warning
```

Each sub-score is min-max normalised across the candidate set, so the weights behave consistently regardless of lane length. Store the sub-scores on the route record — the UI must be able to explain *why* a route ranked first.

3.7 Indicative Total (Milestone 1 placeholder)

```
rate = lane_rates.lookup(origin_region, dest_region, mode, container_type) fallback to regional average if the exact lane is missing
if basis == PER_CONTAINER: base = rate.per_container × container_count
if basis == REVENUE_TON: base = max(rate.per_rt × units, rate.minimum)
if basis == CHARGEABLE_KG: base = max(rate.per_kg × units, rate.minimum)
indicative_total = base × (1 + FLAT_UPLIFT)
FLAT_UPLIFT = 0.35
return { amount, currency, is_indicative: true, basis }
```

The 0.35 uplift is a crude stand-in for fuel, terminal handling, documentation and margin. It exists so the panel shows a plausible number during M1 and gets replaced entirely in M2. Always return `is_indicative: true` and render the label in the UI.

3.8 Orchestration

```
compute_live_estimate(form_state) → LiveEstimate
if not minimally_complete(form_state): return empty_estimate() ← do not throw, show dashes
weights = chargeable_weight(items, mode, load_type)
routes = build_route_options(shipment)
best = routes[0]
transit = estimate_transit(best, mode, ready_date)
total = indicative_total(best, weights)
return { actual_weight, volumetric_weight, chargeable: weights, transit_range: transit.range, arrival_date: transit.arrival, total, route_count: len(routes), is_indicative: true }
```

`minimally_complete` requires origin, destination, mode and at least one item with weight. Below that threshold the panel shows dashes rather than errors — a partially filled form is normal, not a failure state.

4. Reference Data Required

None of the code above runs without this data. Assign it to a named person on day one; it is the critical path for the whole milestone.

Dataset	Rows needed	Source	Columns
Ports	~500 major	UN/LOCODE public dataset	code, name, city, country, lat, lon, is_transshipment_hub, modes[]
Airports	~300 cargo	IATA/ICAO public lists	iata, icao, name, city, country, lat, lon, has_cargo_terminal
Port distances	Top 50 lanes	Published distance tables, manual entry	from_code, to_code, nautical_miles, via (Suez/Panama/Cape)
Carriers	20-30	Manual, from the brokerage	name, scac, service_type, on_time_pct, reliability_score
Carrier services	50-100	Carrier schedule pages	carrier, service_name, port_rotation[], sailings_per_week, transit_days
Container types	8	Standard	code, internal_cbm, max_payload_kg, is_reefer
Lane rate table (M1 stub)	~40 region pairs	Brokerage historical averages	origin_region, dest_region, mode, per_container, per_kg, per_rt, minimum
Business calendars	10 countries	Public holiday lists	country, date, is_holiday
Historical shipments	≥ 500 records	Brokerage records — critical path	lane, carrier, mode, actual_transit_days, shipment_date

If the historical shipment data does not arrive

The transit MAE exit criterion cannot be evidenced without it. Escalate on day 3, not day 8. The fallback is to ship the rule-based transit estimator, evaluate it against carrier-published schedule transit times instead of actuals, and record the criterion as "measured against published schedules pending actuals". State this explicitly at the milestone review rather than quietly changing the definition.

5. Frontend Build List

5.1 Structural Rule

The calculation modules live in `src/lib/pricing/` and import nothing from React. Pages compose; features own logic; `lib/pricing` owns arithmetic. Enforce this with an ESLint import-boundary rule so it survives contact with deadlines.

```
src/
├── app/
│   ├── ship/page.tsx           ← enquiry form route
│   ├── quotes/
│   │   ├── page.tsx           ← quotation dashboard list
│   │   └── [id]/page.tsx      ← quote detail
│   ├── routes/page.tsx        ← route intelligence
│   └── api/
│       ├── gateways/search/route.ts ← async dropdown backend
│       ├── quote/estimate/route.ts ← server-side verify
│       └── shipments/route.ts
├── features/
│   ├── shipment/
│   │   ├── components/
│   │   │   ├── RouteSection.tsx
│   │   │   ├── ServiceTypeSection.tsx
│   │   │   ├── ShipmentItemsSection.tsx
│   │   │   ├── AdditionalSection.tsx
│   │   │   ├── ContactSection.tsx
│   │   │   ├── GatewayAutocomplete.tsx ← build this FIRST
│   │   │   ├── ContainerFields.tsx      ← FCL branch
│   │   │   ├── LooseCargoFields.tsx    ← LCL / air branch
│   │   │   └── HazmatPanel.tsx
│   │   ├── hooks/{useLiveEstimate,useDraftPersistence}.ts
│   │   └── schema/shipmentSchema.ts      ← Zod, shared with backend contract
│   ├── quote/
│   │   └── components/{LiveEstimatePanel,QuoteTable,QuoteStatusBadge}.tsx
│   └── routing/
│       └── components/{RouteMap,RouteOptionCard,TransitBar,
│           CarrierCompareTable,RouteScoreBreakdown}.tsx
├── lib/pricing/                * NO REACT IMPORTS *
│   ├── index.ts                → computeLiveEstimate
│   ├── gateway.ts              → resolveGateway
│   ├── distance.ts             → haversine, road, sea, mainLeg
│   ├── weight.ts               → actual, volumetric, chargeable
│   ├── transit.ts              → estimateTransit, addBusinessDays
│   ├── routeScore.ts           → scoreRoute
│   ├── pricingStub.ts          → indicativeTotal
│   ├── constants.ts            → speeds, divisors, dwells, factors
│   └── types.ts
└── components/{ui,common,maps,charts}/
```

5.2 Pages to Build in Milestone 1

Page	Contains	Done when
/ship	Five-section form, conditional branches, Live Estimate panel, draft persistence	All fields in section 2 present and validated; estimate updates on every change
/quotes	Table with lane, mode, transit, indicative total, status, created date; server-side filters; pagination	Filter by lane, mode, status and date range all work against the API
/quotes/[id]	Shipment facts, three route options, selected route on map, transit breakdown, weight breakdown, score explanation	Every number on screen traces to a documented method
/routes	Global map, lane on-time performance table, transit analytics, port congestion gauges	Renders 200 test lanes without performance degradation

5.3 Component Build Order

Order	Component	Why this position
1	GatewayAutocomplete	Everything depends on structured gateway selection. Nothing else can be correctly tested until this exists.
2	shipmentSchema (Zod)	Defines the contract. Frontend and backend both build against it.
3	lib/pricing modules	Pure functions, unit-testable immediately, no UI dependency. Two developers can work in parallel here.
4	Form sections 1–5	Compose the schema and the autocomplete.

5	LiveEstimatePanel + useLiveEstimate	Wires form state to the pure functions.
6	RouteMap, RouteOptionCard	Needs real route data from the backend to be meaningful.
7	QuoteTable, dashboard pages	Needs persisted records to display.

6. Python Backend Build List

6.1 Module Map

Module	Owns	Milestone 1 deliverable
apps/accounts/	Auth, roles	User model, JWT login/refresh, role field, permission classes for Broker and Admin
apps/masterdata/	Ports, carriers, services	Models with PostGIS point fields, import management commands, search endpoint with trigram matching
apps/customers/	Customer records	Model with tier field, lookup by code
apps/shipments/	Enquiry lifecycle	Model matching the field spec exactly, conditional validation, state machine, CRUD, filters
apps/routing/	Route persistence	Route and RouteLeg models, endpoint returning options for a shipment
apps/quotes/	Quote skeleton	Model with indicative total only. No versioning, no approval — those are M2.
core/	Shared plumbing	Money helpers using Decimal, unit conversions, enums, exception handler, pagination

6.2 Agent Service Modules (FastAPI)

Module	Milestone 1 deliverable
app/agents/route_agent.py	build_route_options, capability filtering, transhipment expansion, structured "not serviced" result
app/agents/state.py	Typed QuoteState dataclass. Even with one agent, establish the pattern now.
app/optimization/route_search.py	Graph over carrier services; direct and one-hop path finding
app/calc/distance.py	haversine, road, sea lookup, main_leg dispatcher
app/calc/weight.py	actual, volumetric, chargeable with three-branch basis
app/calc/transit.py	estimate_transit, add_business_days, mode constants
app/calc/route_score.py	score_route with sub-score persistence
app/calc/pricing_stub.py	indicative_total — deliberately marked temporary in the docstring
app/ml/transit_time/	features.py, train.py, predict.py, baseline.py
app/evaluation/	Route coverage script over 200 lanes; transit MAE script over the hold-out set

6.3 Milestone 1 Endpoints

Method	Endpoint	Purpose
POST	/api/v1/auth/login, /refresh	Token issue and rotation
GET	/api/v1/gateways/search?q=&mode=	Powers the autocomplete. Trigram or prefix match on name, city, code. Returns max 10.
GET	/api/v1/masterdata/container-types	Dropdown source
POST	/api/v1/shipments	Create enquiry, returns 201 with id
GET	/api/v1/shipments	List with filters and cursor pagination
GET	/api/v1/shipments/{id}	Detail with linked routes
POST	/api/v1/shipments/{id}/routes	Trigger the Route Agent, persist and return options
POST	/api/v1/estimate	Stateless server-side estimate for form verification
GET	/api/v1/routes/performance?lane=	On-time percentage by lane and carrier
POST	/v1/route/options (agent svc)	Internal: ranked routings
POST	/v1/estimate/transit (agent svc)	Internal: transit calculation

On duplicating calculation logic in TypeScript and Python

The Live Estimate needs to update instantly as the user types, which means it runs in the browser. The server must independently verify before persisting, which means it runs in Python. Two implementations of the same formulas is a real maintenance cost, and the honest options are: accept it and enforce a shared test-vector file that both implementations must pass, or make the panel call `/api/v1/estimate` with debouncing and accept ~200 ms latency. For Milestone 1, the shared test-vector approach is recommended — a JSON file of 30 input/output pairs that both the Vitest and pytest suites load. When the two drift, a test fails.

7. Transit Time Model

7.1 Two Implementations, One Interface

Build the rule-based estimator first and treat the ML model as an upgrade that must earn its place. Both sit behind `predict_transit(features) → days` and are switched by a feature flag.

Implementation	Method	When it is used
Baseline (rule)	Distance ÷ mode speed + dwell constants, as in section 3.5	Default. Ships regardless of data availability.
ML (LightGBM)	Gradient-boosted regression on historical actuals	Enabled only after it beats the baseline MAE on the hold-out set.

7.2 Feature Set

Feature	Type	Notes
lane_key	Categorical	origin_code + destination_code. Target-encode; one-hot explodes.
distance_nm / km	Numeric	The dominant feature
mode	Categorical	Consider a separate model per mode if data volume allows
carrier_id	Categorical	Carriers differ materially on the same lane
transhipment_count	Integer	Each hop adds dwell and variance
month	Cyclical	Encode as sin/cos, not as an integer 1-12
origin_congestion_index	Numeric	From the port congestion snapshot
dest_congestion_index	Numeric	As above
container_type	Categorical	Reefer and out-of-gauge handle differently
lane_historical_variance	Numeric	Standard deviation of past transits on the lane

7.3 Training and Evaluation Protocol

- 1. Split by time, not randomly.** Train on the older 80%, hold out the most recent 20%. A random split leaks future information about a lane into training and inflates the score.
- 2. Baseline first.** Compute baseline MAE on the hold-out before training anything. This is the number the model must beat.
- 3. Train LightGBM regression.** Objective MAE (not RMSE — outlier shipments should not dominate). Early stopping on a validation slice.
- 4. Report an interval, not a point.** Train quantile models at 0.1 and 0.9 to produce the transit range the UI shows.
- 5. Compare.** If ML MAE is not at least 15% better than baseline, ship the baseline. A marginally better model is not worth the operational cost of maintaining it.
- 6. Record.** Write the evaluation report into the repo, with the model version, feature list, split dates and both MAE figures.

8. Day-by-Day Plan

Day	Backend / Python	Frontend	Data / ML
1	Repo, Docker Compose with Postgres+PostGIS, Django skeleton, settings split	Vite project, Tailwind tokens, layouts, router, ui/ primitives	Source the UN/LOCODE port dataset; request historical shipments — escalate immediately if unavailable
2	Accounts app, JWT, roles. Masterdata models with PostGIS fields	GatewayAutocomplete against a mocked endpoint. Zod schema for all five sections	Clean and load ports and airports. Define the lane key convention
3	/gateways/search with trigram matching. Carriers and services models	Form sections 1 and 2 including conditional load_type and incoterm	Build the port distance table for the top 50 lanes. Load carriers
4	Shipments model matching the field spec, conditional validation, POST and GET	Section 3 with the FCL/LCL conditional branch. Section 4 hazmat panel	Load carrier services with port rotations and sailing frequency
5	FastAPI skeleton, service auth, calc/distance.py and calc/weight.py with unit tests	lib/pricing/ distance and weight modules with Vitest tests. Shared test-vector file agreed	Baseline transit estimator implemented and MAE measured on whatever data exists
6	calc/transit.py with business calendars. route_search.py graph over services	Section 5, draft persistence, progress indicator, mobile layout collapse	Feature engineering for the transit model. Time-based split
7	route_agent.py: capability filters, transshipment expansion, scoring. /v1/route/options	LiveEstimatePanel wired to useLiveEstimate. Indicative labelling	Train LightGBM. Compare against baseline. Decide which ships
8	Routing app persistence. /shipments/{id}/routes. /estimate verification endpoint	RouteMap with MapLibre, RouteOptionCard, TransitBar, score breakdown	Run the 200-lane coverage script. Fix gaps in service data
9	Quotes skeleton model, indicative total persistence, list and detail endpoints with filters	Quotation dashboard list with filters, quote detail page, carrier comparison table	Final evaluation run. Write the evaluation report into the repo
10	Contract tests, fix defects, seed demo data	Playwright journey, empty and loading states, accessibility pass	Milestone evidence pack: coverage output, MAE report, demo script

The two days that decide the milestone

Day 2 (GatewayAutocomplete) and Day 5 (pure calculation modules with tests). If both land on time, days 6–10 are assembly. If either slips, the Route Agent has nothing solid to build on and the coverage script cannot run. Protect these two days from meetings and side requests.

9. Definition of Done Checklist

9.1 Functional

☐	Item
☐	All fields in section 2 exist in the form, including the shaded additions, with correct conditional visibility
☐	Origin and destination are structured gateway selections, not free text
☐	FCL and LCL/air branches show different field sets; Container never shows dimensions
☐	Live Estimate updates on every relevant field change and shows dashes when incomplete
☐	Chargeable weight displays the correct basis label for all three branches
☐	Transit shows a range and an estimated arrival date respecting business calendars
☐	Indicative total is labelled as indicative in the UI
☐	Submitting creates a shipment record and returns route options within 8 seconds p95
☐	Quote detail renders the selected route on a map with a score breakdown
☐	Dashboard filters work server-side against lane, mode, status and date range
☐	Unserviced lanes return a structured explanation, not an empty list or an error

9.2 Evidence Required at Review

☐	Item
☐	Coverage script output: $\geq 98\%$ of 200 test lanes return ≥ 2 viable routings
☐	Transit evaluation report: baseline MAE, ML MAE, which one shipped and why
☐	Live demo: enquiry $\rightarrow$ routes on map $\rightarrow$ detail $\rightarrow$ filtered list, performed by someone who did not build it

9.3 Engineering Quality

☐	Item
☐	lib/pricing/ and app/calc/ contain zero framework imports
☐	Shared test-vector file passes in both the Vitest and pytest suites
☐	All monetary values use Decimal in Python; no float appears in any calculation path
☐	Unit test coverage $\geq 80\%$ on the calculation modules specifically
☐	OpenAPI contract published and matching the implemented endpoints
☐	Server-side /estimate independently recomputes rather than trusting client values
☐	Reference data loaded via repeatable management commands, not manual SQL
☐	ESLint import-boundary rule active on lib/pricing

9.4 Carry-Forward Register

Anything not completed is written here with an owner and a target milestone, rather than silently absorbed. Review this list at the start of Milestone 2 planning.

Item	Owner	Target	Blocker